

OpenVAS — Study Guide

OpenVAS (now branded **Greenbone Vulnerability Management / GVM**) is the leading **free, open-source** alternative to Nessus — same core purpose (automated vulnerability scanning), fully open-source stack, and a genuinely strong choice for a portfolio project since you can self-host and demonstrate the entire pipeline without licensing constraints.

1. OpenVAS vs. Nessus — Why Both Exist

	Nessus	OpenVAS/GVM
License	Commercial (free tier limited to 16 IPs)	Fully open-source, no IP limits
Plugin/feed freshness	Very fast (commercial funding)	Slightly slower feed updates (community-driven)
Ease of setup	Simpler installer	More involved setup (multiple components)
Report polish	Generally more polished UI/reports	Functional, less polished but fully capable
Best for	Professional engagements, client-facing reports	Home lab, learning, budget-constrained orgs, full self-hosted control

For your GitHub portfolio specifically, OpenVAS/GVM is arguably the stronger choice to showcase — it demonstrates you can deploy and operate a full open-source vulnerability management stack, not just run someone else's SaaS product.

2. Architecture — What's Actually Running

OpenVAS/GVM isn't one binary — it's a stack of components:

```
Scanner (openvas-scanner)    → runs the actual NVTs (Network Vulnerability Tests)
Manager (gvmd)               → coordinates scans, stores results in a database
Web UI (Greenbone Security Assistant) → the interface you actually interact with
Feed                         → the vulnerability test database, updated regularly
```

Worth understanding this because when something breaks (a common experience on first setup), knowing which component is failing saves a lot of debugging time.

3. Installation (Kali — Simplest Path)

```
sudo apt update && sudo apt install openvas -y
sudo gvm-setup      # downloads feeds, sets up database - takes a while first time
sudo gvm-check-setup  # verifies everything installed correctly
sudo gvm-start      # starts all services
```

Access the web UI at `https://localhost:9392` with the admin credentials generated during setup (`gvm-setup` prints these — save them immediately, they're shown once).

Note: initial feed sync can take a genuinely long time (sometimes hours) since it's downloading the full NVT database — worth starting this before you plan to actually use it.

4. Basic Workflow (Web UI)

1. Log into Greenbone Security Assistant (`https://localhost:9392`)
2. Configuration → Targets → New Target (add your test IP/range)
3. Scans → Tasks → New Task → select your target + a scan config
4. Start the task
5. Review results under Scans → Results, sorted by severity
6. Export report (PDF, CSV, XML)

5. Scan Configs (Templates)

Config	Use Case
Full and fast	Good general-purpose default
Full and very deep	More thorough, much slower
Discovery	Host/service discovery only, no vuln checks
Host Discovery	Just finds live hosts
System Discovery	Deeper host/OS fingerprinting

6. Credentialed Scanning in OpenVAS

Same concept as Nessus — adding credentials dramatically improves accuracy:

```
Configuration → Credentials → New Credential → choose type: SSH (Linux), SMB (Windows), ESXi, SNMP → attach the credential to your Target
```

7. NVTs (Network Vulnerability Tests) — OpenVAS's Equivalent of Nessus Plugins

Each check OpenVAS runs is an **NVT**, written in OpenVAS's own scripting language (NASL — Nessus Attack Scripting Language, historically shared lineage with Nessus before the fork). NVTs are organized into families:

```
Buffer overflow
Denial of Service
General
Web application abuses
Windows
SMTP problems
```

You can enable/disable entire families in a custom scan config — useful for scoping a scan (e.g., disabling **Denial of Service** family checks in a production environment where you don't want to risk crashing anything).

8. Reading OpenVAS Results

Findings include:

- **QoD (Quality of Detection)** — OpenVAS's own confidence score for how certain it is the finding is accurate (a feature Nessus doesn't have as explicitly) — higher QoD = trust the finding more, lower QoD = worth manual verification first
- **CVSS score**
- **Solution type** — Workaround, Mitigation, VendorFix, NoneAvailable
- **References** — CVE IDs, vendor advisories

The **QoD score** is genuinely useful and worth highlighting in your own reports — it's a built-in signal for exactly the "don't blindly trust scanner output" principle covered in the Nessus guide.

9. CLI Usage (`gvm-cli` / `gvm-tools`)

For automation/scripting (relevant since you already build Python-based tools like SmartScan):

```
pip install gvm-tools
gvm-cli --gmp-username admin --gmp-password 'yourpass' socket
t --xml "<get_tasks/>"
```

This opens the door to **integrating OpenVAS scan triggering/result retrieval directly into your own Python tools** — genuinely a strong SmartScan v3 feature idea (automated scan orchestration + result parsing, similar pattern to how you integrated the NVD API).

10. Practical Lab Example

1. Install OpenVAS on Kali, complete gvm-setup
2. Add Metasploitable 2 as a target
3. Run a "Full and fast" scan, uncredentialed
4. Add SSH credentials (msfadmin/msfadmin), re-run credentialed
5. Compare QoD scores and finding counts between runs
6. Cross-reference top findings against your earlier manual Nmap/NSE results

11. Kill Chain / ATT&CK Mapping

Framework	Mapping
Cyber Kill Chain	Reconnaissance
MITRE ATT&CK	T1595.002 (Active Scanning: Vulnerability Scanning)

Quick Reference Cheat Sheet

```
sudo apt install openvas
sudo gvm-setup
sudo gvm-check-setup
```

```
sudo gvm-start  
# Web UI: https://localhost:9392  
  
pip install gvm-tools      # CLI automation
```
